

Báo cáo bài tập Lập trình mạng

Giảng viên hướng dẫn: Lê Bá Vui

Họ và tên sinh viên: Ngô Hoàng Minh

Mã số sinh viên: 20235377

Học phần: IT4060

Mã lớp: 168503

Bài tập 1: Tạo trang web thực hiện các phép tính cộng, trừ, nhân, chia và trả lại kết quả cho trình duyệt. Nhận tham số từ lệnh GET, tham số từ lệnh POST, các tham số bao gồm toán tử (cộng, trừ, nhân, chia) và 2 toán hạng, server trả lại kết quả trang web hiển thị phép tính

Mã nguồn:

```
int main() {
    int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (listener == -1) {
        perror("socket() failed");
        return 1;
    }

    if (setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &(int){1}, sizeof(int))) {
        perror("setsockopt() failed");
        close(listener);
        return 1;
    }

    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(7000);

    if (bind(listener, (struct sockaddr *)&addr, sizeof(addr))) {
        perror("bind() failed");
        close(listener);
        return 1;
    }

    if (listen(listener, 5)) {
        perror("listen() failed");
        close(listener);
        return 1;
    }

    // Server is now listening for incoming connections
    printf("Server is listening on port 7000...\n");
}
```

khởi tạo socket lắng nghe kết nối ở cổng 7000

```
signal(SIGCHLD, signal_handler);

while (1) {
    int client = accept(listener, NULL, NULL);
    //printf("New client connected: %d\n", client);
    if (fork() == 0) {
        // Xu ly trong tien trinh con
        close(listener);
        handle_calc_logic(client);
        exit(0);
    }
    // Xu ly o tien trinh cha
    close(client);
}

close(listener);
return 0;
}
```

hàm fork() tạo tiến trình con xử lý yêu cầu client

```

void handle_calc_logic(int client_fd) {
    char raw_data[4096] = {0};
    recv(client_fd, raw_data, sizeof(raw_data) - 1, 0);

    char http_method[16], path_info[256];
    sscanf(raw_data, "%s %s", http_method, path_info);

    char operator[16] = "";
    double val1 = 0, val2 = 0;
    int is_valid = 0;

    if (strcmp(http_method, "GET") == 0) {
        char *q_ptr = strchr(path_info, '?');
        if (q_ptr && sscanf(q_ptr + 1, "op=%[^&]&a=%lf&b=%lf", operator, &val1, &val2) == 3)
            is_valid = 1;
    }
    else if (strcmp(http_method, "POST") == 0) {
        char *body_ptr = strstr(raw_data, "\r\n\r\n");
        if (body_ptr && sscanf(body_ptr + 4, "op=%[^&]&a=%lf&b=%lf", operator, &val1, &val2) == 3)
            is_valid = 1;
    }
}

```

hàm xử lý yêu cầu của client: lấy request của client, xác định phương thức, tham số, tính hợp lệ của cú pháp

```

char html_output[1024];
if (is_valid) {
    double calc_res = 0;
    char symbol = '?';
    if (strcmp(operator, "add") == 0) { calc_res = val1 + val2; symbol = '+'; }
    else if (strcmp(operator, "sub") == 0) { calc_res = val1 - val2; symbol = '-'; }
    else if (strcmp(operator, "mul") == 0) { calc_res = val1 * val2; symbol = '*'; }
    else if (strcmp(operator, "div") == 0 && val2 != 0) { calc_res = val1 / val2; symbol = '/'; }

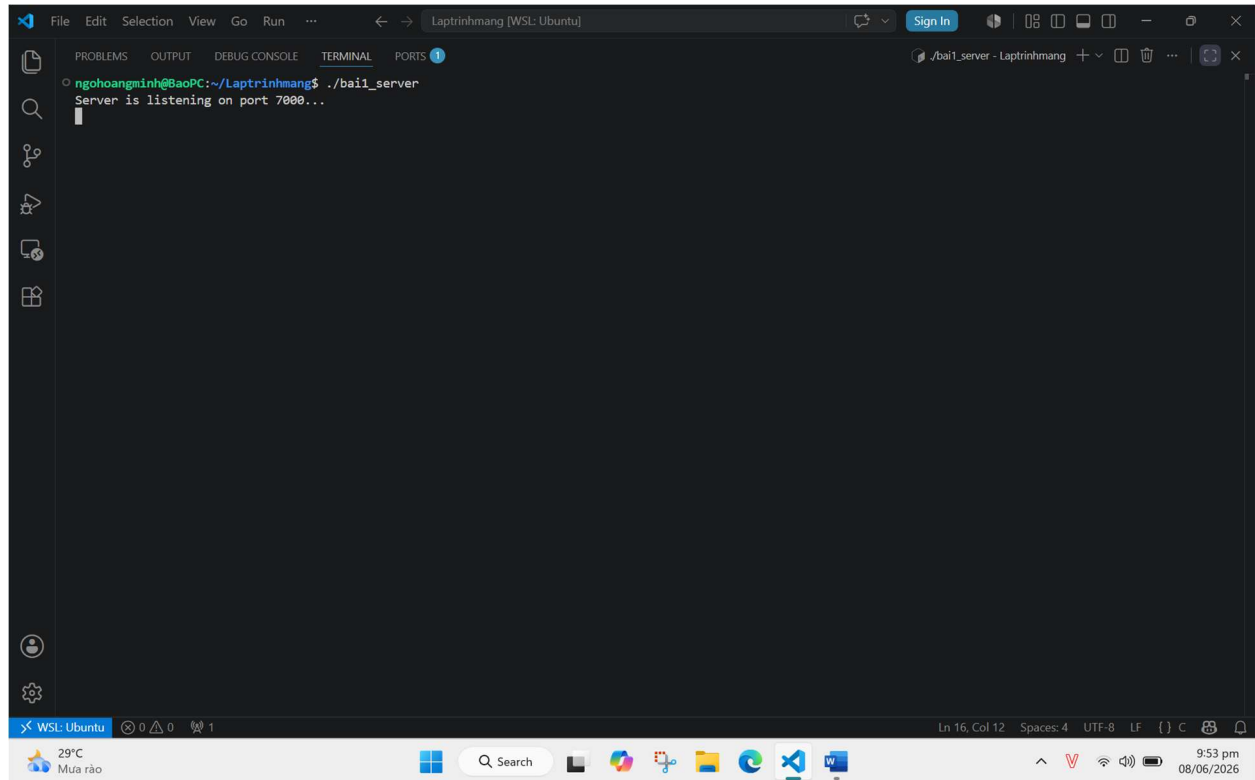
    snprintf(html_output, sizeof(html_output), "<html><body><h1>Ket qua: %.2g %c %.2g = %.2g</h1><
} else {
    snprintf(html_output, sizeof(html_output), "<html><body><h2>Thong bao: Sai cu phap. Format: ?o
}

char final_response[2048];
snprintf(final_response, sizeof(final_response), "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n");
send(client_fd, final_response, strlen(final_response), 0);
close(client_fd);
}

```

hàm xử lý yêu cầu client: tính toán và trả lại kết quả cho client

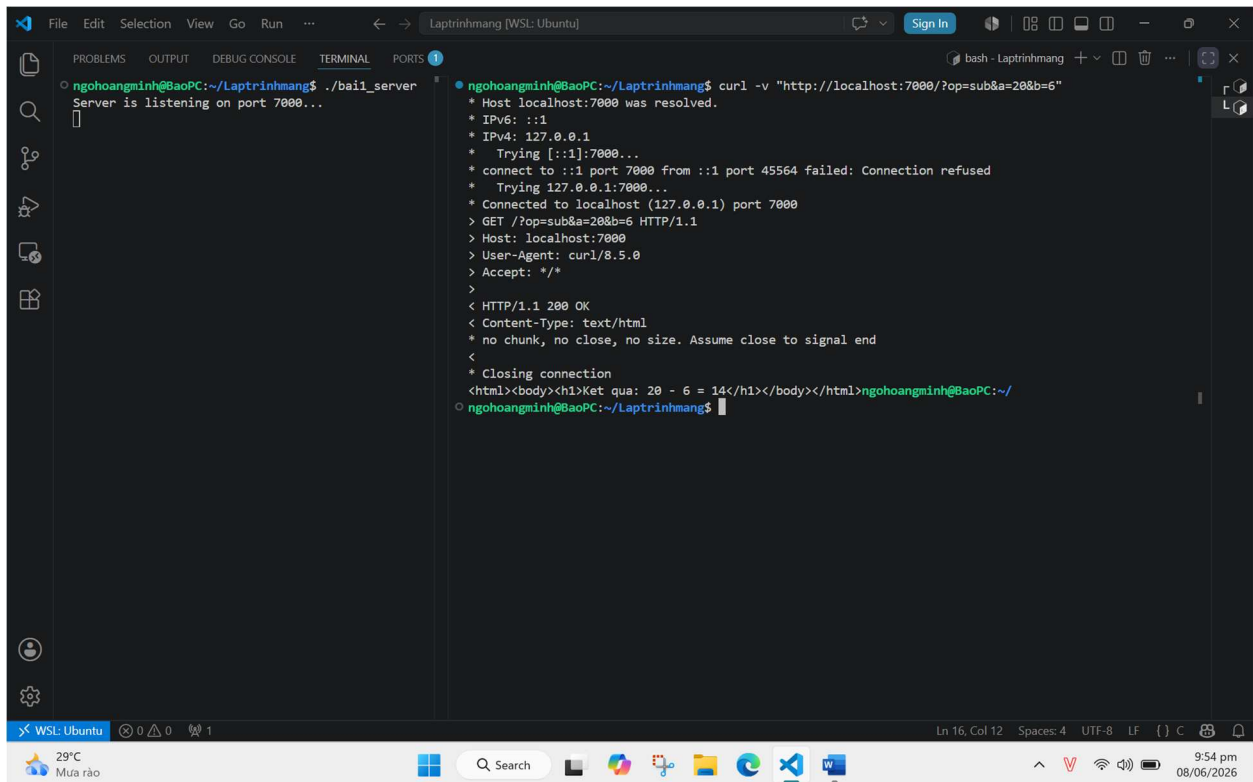
Kết quả chạy:



The image shows a screenshot of a Visual Studio Code (VS Code) window with a terminal open. The terminal is titled "Laptrinhmang [WSL: Ubuntu]". The prompt is "ngohoangminh@BaoPC:~/Laptrinhmang\$". The command entered is "./bai1_server". The output is "Server is listening on port 7000...". The terminal window has a dark theme. The VS Code interface includes a sidebar on the left with icons for Explorer, Search, Source Control, Run and Debug, and Extensions. The top menu bar shows File, Edit, Selection, View, Go, Run, and a "Sign In" button. The bottom status bar shows "WSL: Ubuntu", "Ln 16, Col 12", "Spaces: 4", "UTF-8", "LF", and "C". The Windows taskbar at the bottom shows the date and time as "9:53 pm 08/06/2026" and the weather as "29°C Mưa rào".

```
ngohoangminh@BaoPC:~/Laptrinhmang$ ./bai1_server
Server is listening on port 7000...
```

khởi chạy server lắng nghe kết nối ở cổng 7000

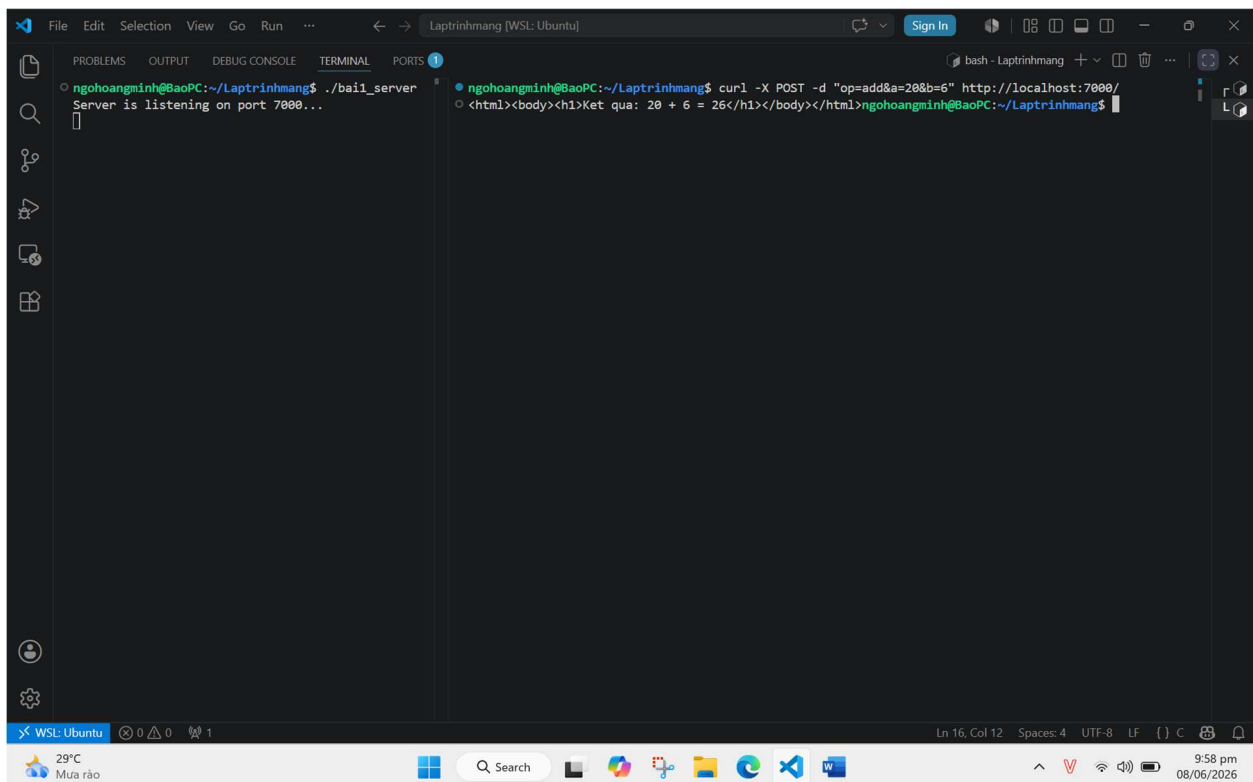


The screenshot shows a terminal window with two panes. The left pane shows a command prompt where the user has run `./bai1_server`, and the output is "Server is listening on port 7000...". The right pane shows the output of a `curl -v` command. The output details the connection process to `localhost:7000`, showing the HTTP 1.1 200 OK response and the HTML body content: `<html><body><h1>Ket qua: 20 - 6 = 14</h1></body></html>`.

```
ngohoangminh@BaoPC:~/Laptrinhmang$ ./bai1_server
Server is listening on port 7000...

ngohoangminh@BaoPC:~/Laptrinhmang$ curl -v "http://localhost:7000/?op=sub&a=20&b=6"
* Host localhost:7000 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:7000...
* connect to ::1 port 7000 from ::1 port 45564 failed: Connection refused
* Trying 127.0.0.1:7000...
* Connected to localhost (127.0.0.1) port 7000
> GET /?op=sub&a=20&b=6 HTTP/1.1
> Host: localhost:7000
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Content-Type: text/html
* no chunk, no close, no size. Assume close to signal end
<
* Closing connection
<html><body><h1>Ket qua: 20 - 6 = 14</h1></body></html>ngohoangminh@BaoPC:~/Laptrinhmang$
```

request GET với phép trừ 20 và 6

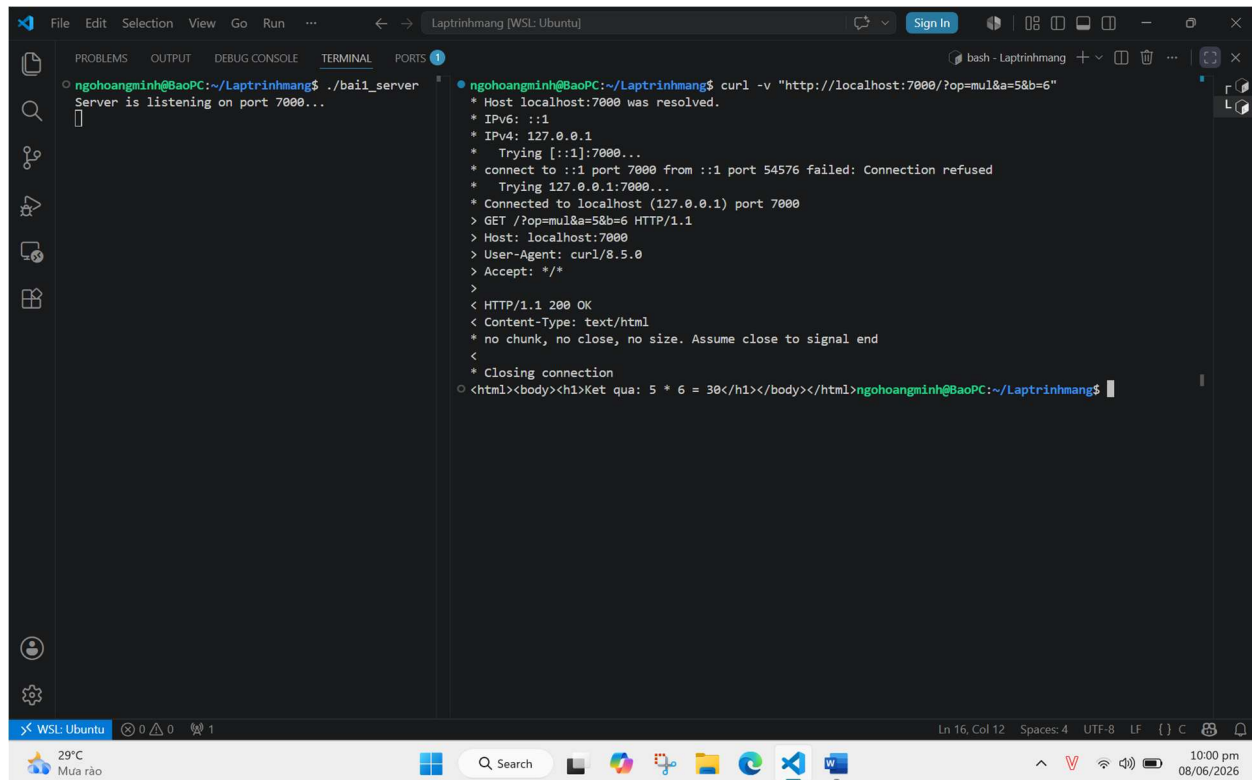


The screenshot shows a terminal window with two panes. The left pane shows the same server output as the previous image. The right pane shows the output of a `curl -X POST` command. The output details the connection process to `localhost:7000`, showing the HTTP 1.1 200 OK response and the HTML body content: `<html><body><h1>Ket qua: 20 + 6 = 26</h1></body></html>`.

```
ngohoangminh@BaoPC:~/Laptrinhmang$ ./bai1_server
Server is listening on port 7000...

ngohoangminh@BaoPC:~/Laptrinhmang$ curl -X POST -d "op=add&a=20&b=6" http://localhost:7000/
<html><body><h1>Ket qua: 20 + 6 = 26</h1></body></html>ngohoangminh@BaoPC:~/Laptrinhmang$
```

request POST với phép cộng 20 và 6

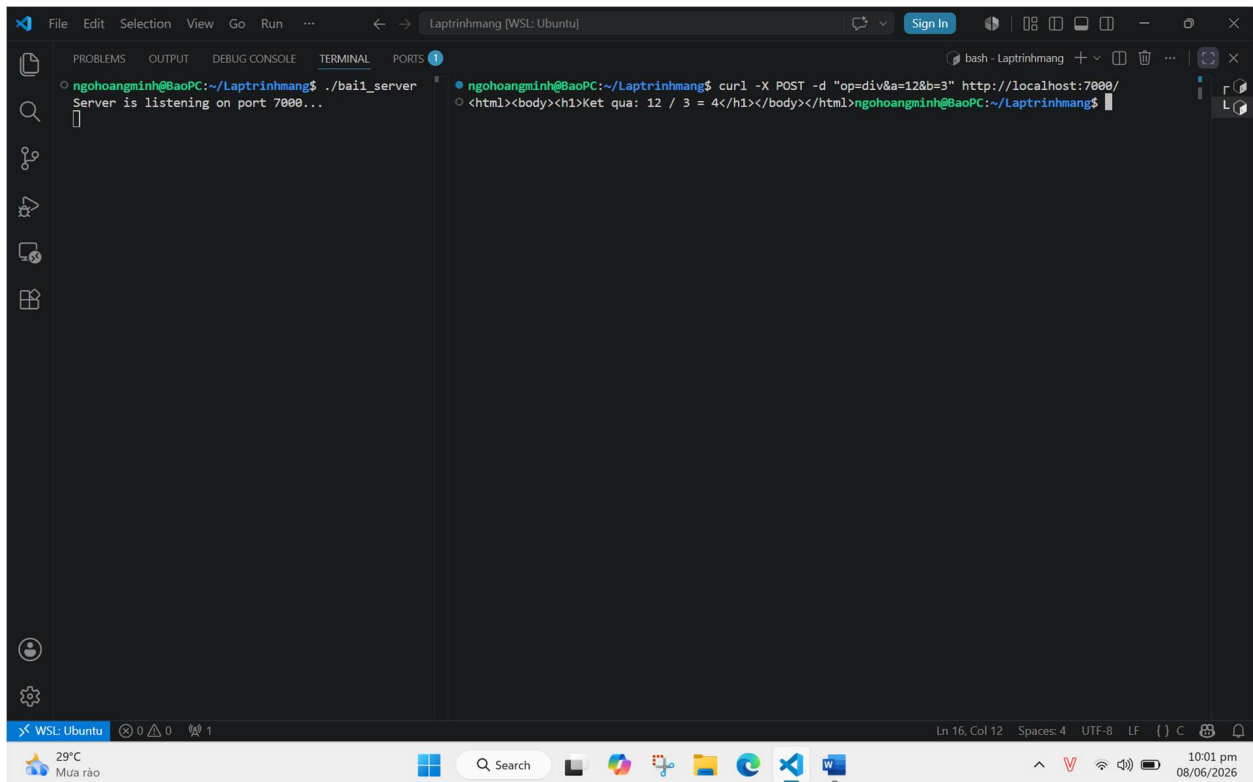


The screenshot shows a Visual Studio Code editor window with a terminal pane open. The terminal is running on a WSL Ubuntu environment. On the left, a file explorer shows a directory named 'Laptrinhmang' containing a file 'bai1_server'. The terminal output shows the server listening on port 7000. On the right, a terminal window shows the execution of a curl command to a local host on port 7000 with query parameters. The output shows a successful GET request with a 200 OK status and a text/html content type. The response body contains the text 'Ket qua: 5 * 6 = 30'.

```
ngohoangminh@BaoPC:~/Laptrinhmang$ ./bai1_server
Server is listening on port 7000...

ngohoangminh@BaoPC:~/Laptrinhmang$ curl -v "http://localhost:7000/?op=mul&a=5&b=6"
* Host localhost:7000 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:7000...
* connect to ::1 port 7000 from ::1 port 54576 failed: Connection refused
* Trying 127.0.0.1:7000...
* Connected to localhost (127.0.0.1) port 7000
> GET /?op=mul&a=5&b=6 HTTP/1.1
> Host: localhost:7000
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Content-Type: text/html
* no chunk, no close, no size. Assume close to signal end
<
* Closing connection
<html><body><h1>Ket qua: 5 * 6 = 30</h1></body></html>ngohoangminh@BaoPC:~/Laptrinhmang$
```

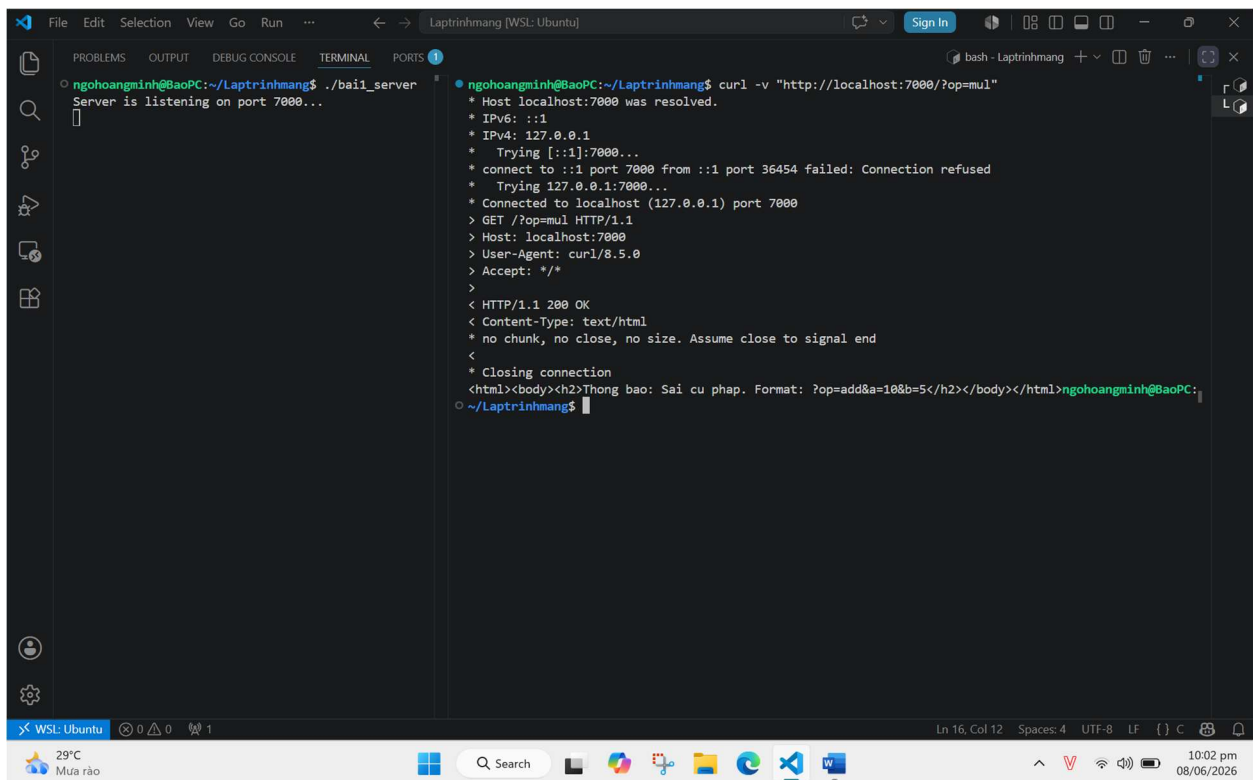
request GET với phép nhân 5 và 6



The screenshot shows a VS Code window with a terminal titled "Laptrinhmang [WSL: Ubuntu]". The terminal has two panes. The left pane shows the command `ngohoangminh@BaoPC:~/Laptrinhmang$ ./bai1_server` and the output "Server is listening on port 7000...". The right pane shows the command `ngohoangminh@BaoPC:~/Laptrinhmang$ curl -X POST -d "op=div&a=12&b=3" http://localhost:7000/` and the output `<html><body><h1>Ket qua: 12 / 3 = 4</h1></body></html>`. The status bar at the bottom indicates "WSL: Ubuntu" and the system clock shows 10:01 pm on 08/06/2026.

```
File Edit Selection View Go Run ... Laptrinhmang [WSL: Ubuntu]
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ngohoangminh@BaoPC:~/Laptrinhmang$ ./bai1_server
Server is listening on port 7000...
ngohoangminh@BaoPC:~/Laptrinhmang$ curl -X POST -d "op=div&a=12&b=3" http://localhost:7000/
<html><body><h1>Ket qua: 12 / 3 = 4</h1></body></html>ngohoangminh@BaoPC:~/Laptrinhmang$
WSL: Ubuntu 29°C Mưa rào 10:01 pm 08/06/2026
```

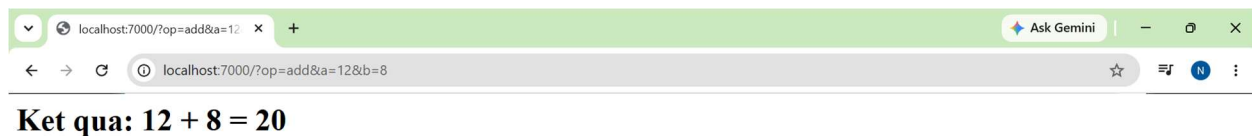
request POST với phép chia 12 và 3



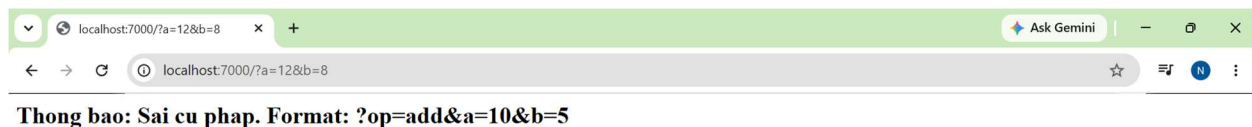
The screenshot shows a VS Code window with a terminal titled "Laptrinhmang [WSL: Ubuntu]". The terminal has two panes. The left pane shows the command `ngohoangminh@BaoPC:~/Laptrinhmang$ ./bai1_server` and the output "Server is listening on port 7000...". The right pane shows the command `ngohoangminh@BaoPC:~/Laptrinhmang$ curl -v "http://localhost:7000/?op=mul"` and the output showing a connection refused error. The status bar at the bottom indicates "WSL: Ubuntu" and the system clock shows 10:02 pm on 08/06/2026.

```
File Edit Selection View Go Run ... Laptrinhmang [WSL: Ubuntu]
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ngohoangminh@BaoPC:~/Laptrinhmang$ ./bai1_server
Server is listening on port 7000...
ngohoangminh@BaoPC:~/Laptrinhmang$ curl -v "http://localhost:7000/?op=mul"
* Host localhost:7000 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:7000...
* connect to ::1 port 7000 from ::1 port 36454 failed: Connection refused
* Trying 127.0.0.1:7000...
* Connected to localhost (127.0.0.1) port 7000
> GET /?op=mul HTTP/1.1
> Host: localhost:7000
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Content-Type: text/html
* no chunk, no close, no size. Assume close to signal end
* Closing connection
<html><body><h2>Thong bao: Sai cu phap. Format: ?op=add&a=10&b=5</h2></body></html>ngohoangminh@BaoPC:~/Laptrinhmang$
WSL: Ubuntu 29°C Mưa rào 10:02 pm 08/06/2026
```

request với cú pháp sai



hiển thị request với phép cộng giữa 12 và 8 trên trình duyệt



thông báo lỗi trên trình duyệt

Bài tập 2: Lập trình ứng dụng máy chủ HTTP file với các chức năng: Khi trình duyệt yêu cầu thư mục gốc, trả về nội dung là trang web chứa danh sách các liên kết thư mục và tập tin của thư mục hiện tại trên server, in đậm đối với thư mục và in nghiêng đối với file. Nếu người dùng ấn vào liên kết thư mục, server trả về nội dung của thư mục con đó. Nếu nhấn vào liên kết file, trả về nội dung file. Hỗ trợ các file văn bản, audio, video, ảnh.

Mã nguồn:

```
int main() {
    int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (listener == -1) {
        perror("socket() failed");
        return 1;
    }

    if (setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &(int){1}, sizeof(int))) {
        perror("setsockopt() failed");
        close(listener);
        return 1;
    }

    struct sockaddr_in addr = {0};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(7000);

    if (bind(listener, (struct sockaddr *)&addr, sizeof(addr))) {
        perror("bind() failed");
        close(listener);
        return 1;
    }

    if (listen(listener, 5)) {
        perror("listen() failed");
        close(listener);
        return 1;
    }

    // Server is now listening for incoming connections
    printf("Server is listening on port 7000...\n");
}
```

khởi tạo socket lắng nghe ở cổng 7000

```

signal(SIGCHLD, signal_handler);

while (1) {
    int client = accept(listener, NULL, NULL);
    if (fork() == 0) {
        // Xu ly trong tien trinh con
        close(listener);

        char req[2048], method[16], uri[256], path[512] = ".";
        recv(client, req, sizeof(req), 0);
        sscanf(req, "%s %s", method, uri);

        clean_url(path + 1, uri);

        struct stat st;
        if (stat(path, &st) == 0 && S_ISDIR(st.st_mode))
            list_directory(client, path, uri);
        else
            stream_file(client, path);
        close(client);
        exit(0);
    }
    // Xu ly o tien trinh cha
    close(client);
}

close(listener);
return 0;
}

```

khởi tạo tiến trình con bằng fork() xử lý client, nhận request của client, xử lý url sau đó hiển thị danh sách file của thư mục hoặc dữ liệu file cho client

```

void clean_url(char *dest, const char *src) {
    while (*src) {
        if (*src == '%' && src[1] && src[2]) {
            *dest++ = ' '; src += 3;
        } else if (*src == '+') {
            *dest++ = ' '; src++;
        } else {
            *dest++ = *src++;
        }
    }
    *dest = '\0';
}

```

hàm xử lý url

```

void list_directory(int client, const char *dir_path, const char *uri) {
    DIR *d = opendir(dir_path);
    if (!d) return;

    send(client, "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n<html><body>", 55, 0);

    struct dirent *entry;
    while ((entry = readdir(d)) != NULL) {
        if (strcmp(entry->d_name, ".") == 0) continue;

        char full[1024], link[1024];
        snprintf(full, sizeof(full), "%s/%s", dir_path, entry->d_name);
        snprintf(link, sizeof(link), "%s%s", uri, (strcmp(uri, "/") == 0 ? "" : "/"), entry->d_name);

        struct stat st;
        stat(full, &st);

        if (S_ISDIR(st.st_mode))
            dprintf(client, "<b><a href='%s'>%s</a></b><br>", link, entry->d_name);
        else
            dprintf(client, "<i><a href='%s'>%s</a></i><br>", link, entry->d_name);
    }
    send(client, "</body></html>", 14, 0);
    closedir(d);
}

```

hàm liệt kê danh sách file trong thư mục và đường dẫn đến file đó: vào thư mục gốc, duyệt qua danh sách thành phần trong thư mục, rồi hiển thị cho client

```

void stream_file(int client, const char *path) {
    FILE *fp = fopen(path, "rb");
    if (!fp) {
        char *err = "HTTP/1.1 404 Not Found\r\n\r\nFile Not Found";
        send(client, err, strlen(err), 0);
        return;
    }

    char head[256];
    snprintf(head, sizeof(head), "HTTP/1.1 200 OK\r\nContent-Type: %s\r\n\r\n", get_content_type(path));
    send(client, head, strlen(head), 0);

    char chunk[8192];
    size_t len;
    while ((len = fread(chunk, 1, sizeof(chunk), fp)) > 0) send(client, chunk, len, 0);
    fclose(fp);
}

```

hàm hiển thị nội dung file: open file theo đường dẫn, trả về lỗi nếu không tìm thấy file hay nội dung file nếu tìm thấy

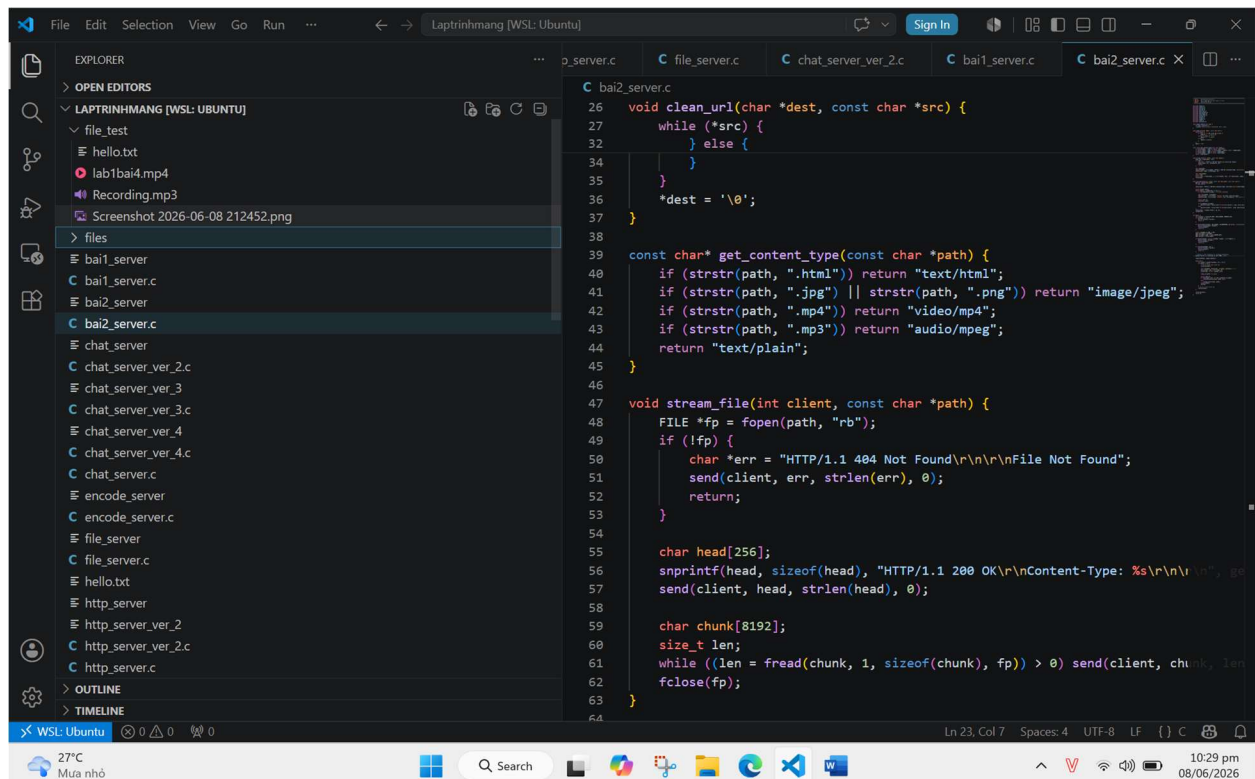
```

const char* get_content_type(const char *path) {
    if (strstr(path, ".html")) return "text/html";
    if (strstr(path, ".jpg") || strstr(path, ".png")) return "image/jpeg";
    if (strstr(path, ".mp4")) return "video/mp4";
    if (strstr(path, ".mp3")) return "audio/mpeg";
    return "text/plain";
}

```

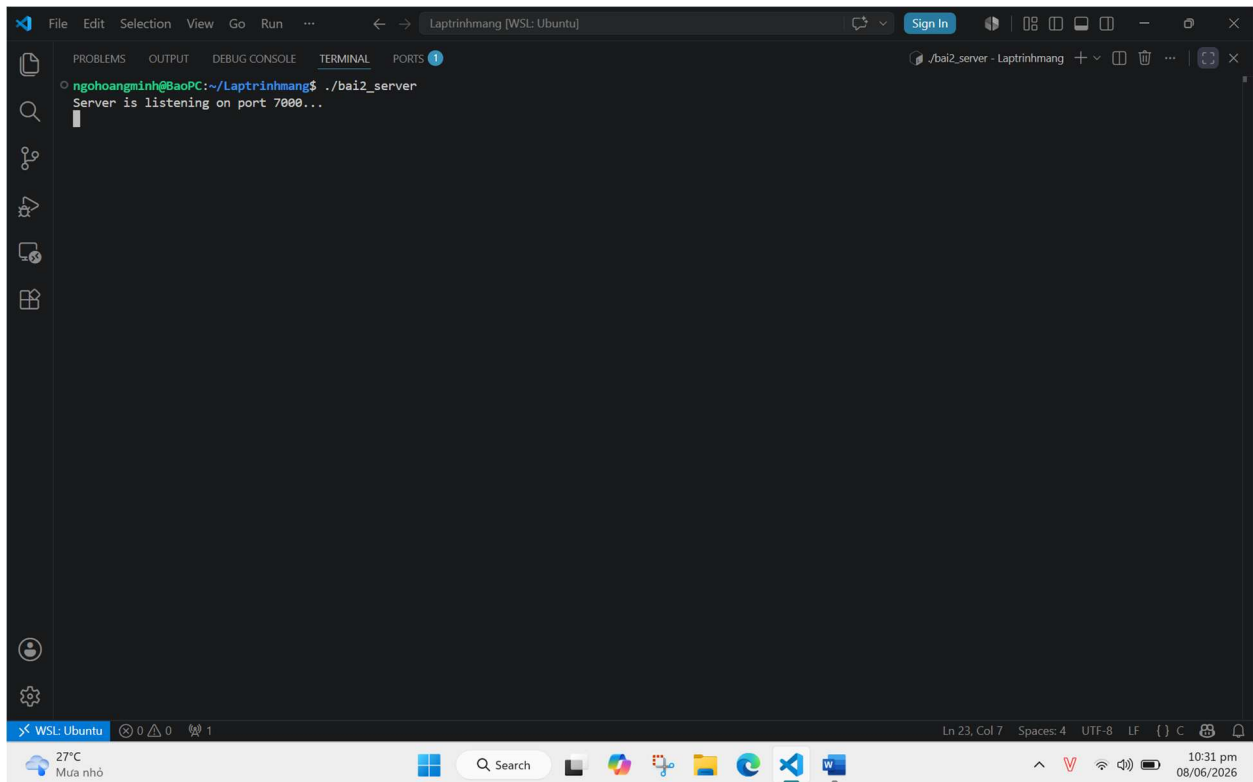
hàm xác định kiểu tệp: tùy vào định dạng file trả kiểu: html kiểu text/html, .png hay .jpg kiểu image/jpeg, .mp4 kiểu video/mp4, .mp3 kiểu audio.mp3

Kết quả chạy:



```
26 void clean_url(char *dest, const char *src) {
27     while (*src) {
32     } else {
34     }
35 }
36 *dest = '\0';
37 }
38
39 const char* get_content_type(const char *path) {
40     if (strstr(path, ".html")) return "text/html";
41     if (strstr(path, ".jpg") || strstr(path, ".png")) return "image/jpeg";
42     if (strstr(path, ".mp4")) return "video/mp4";
43     if (strstr(path, ".mp3")) return "audio/mpeg";
44     return "text/plain";
45 }
46
47 void stream_file(int client, const char *path) {
48     FILE *fp = fopen(path, "rb");
49     if (!fp) {
50         char *err = "HTTP/1.1 404 Not Found\r\n\r\nFile Not Found";
51         send(client, err, strlen(err), 0);
52         return;
53     }
54
55     char head[256];
56     snprintf(head, sizeof(head), "HTTP/1.1 200 OK\r\nContent-Type: %s\r\n\r\n", get_content_type(path));
57     send(client, head, strlen(head), 0);
58
59     char chunk[8192];
60     size_t len;
61     while ((len = fread(chunk, 1, sizeof(chunk), fp)) > 0) send(client, chunk, len, 0);
62     fclose(fp);
63 }
64 }
```

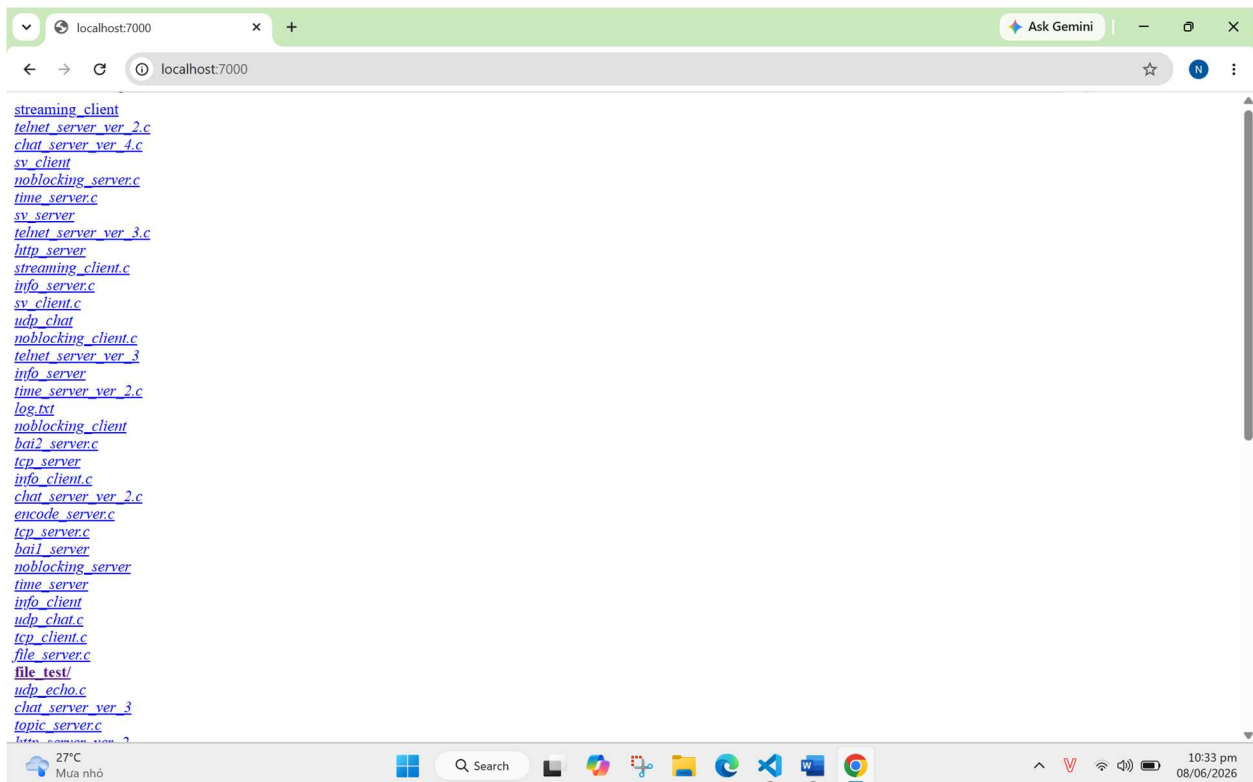
thư mục gốc: gồm file mã nguồn và folder file_test chứa các file có định dạng .txt, .mp3, .mp4, .png



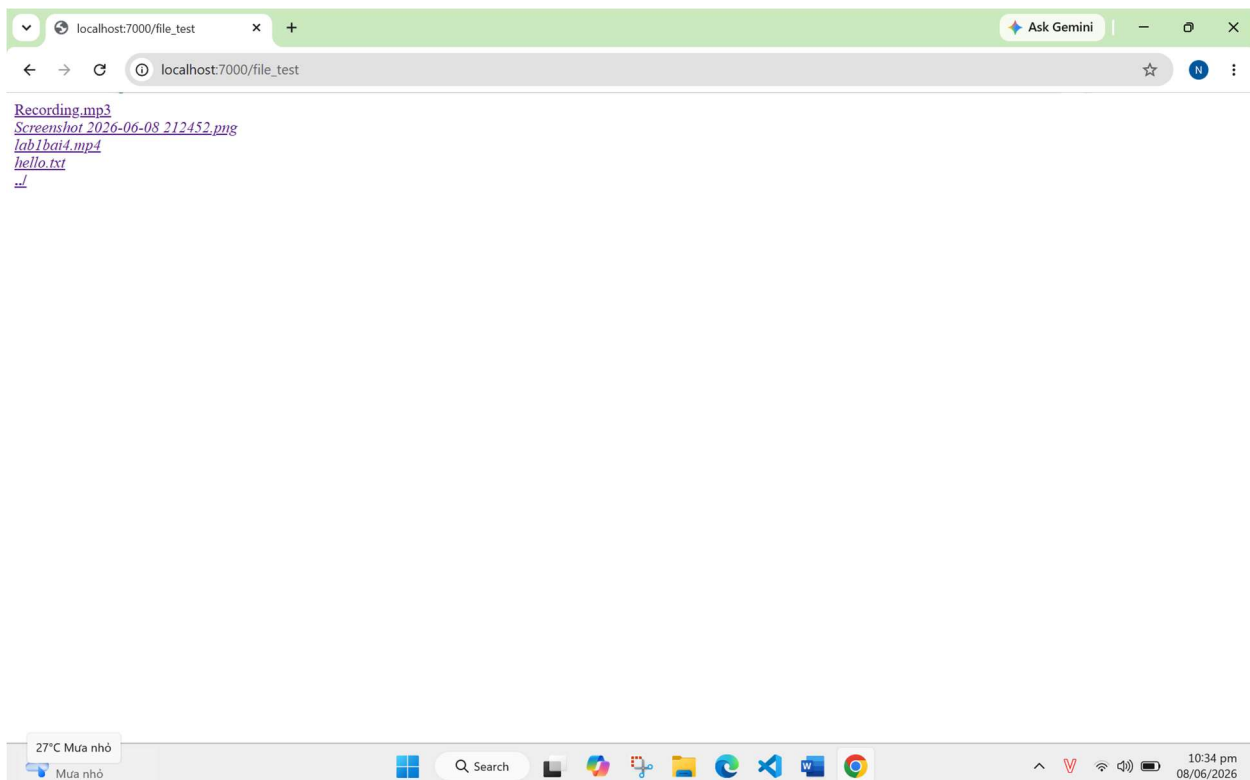
The screenshot shows a VS Code editor window with a terminal pane at the bottom. The terminal is running a command prompt for a user named 'ngochoangminh' on a machine named 'Laptrinhmang'. The command executed is './bai2_server', which outputs 'Server is listening on port 7000...'. The terminal window has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The status bar at the bottom indicates the file is 'bai2_server - Laptrinhmang' and the terminal is running on 'WSL: Ubuntu'.

```
ngochoangminh@Laptrinhmang:~/Laptrinhmang$ ./bai2_server
Server is listening on port 7000...
```

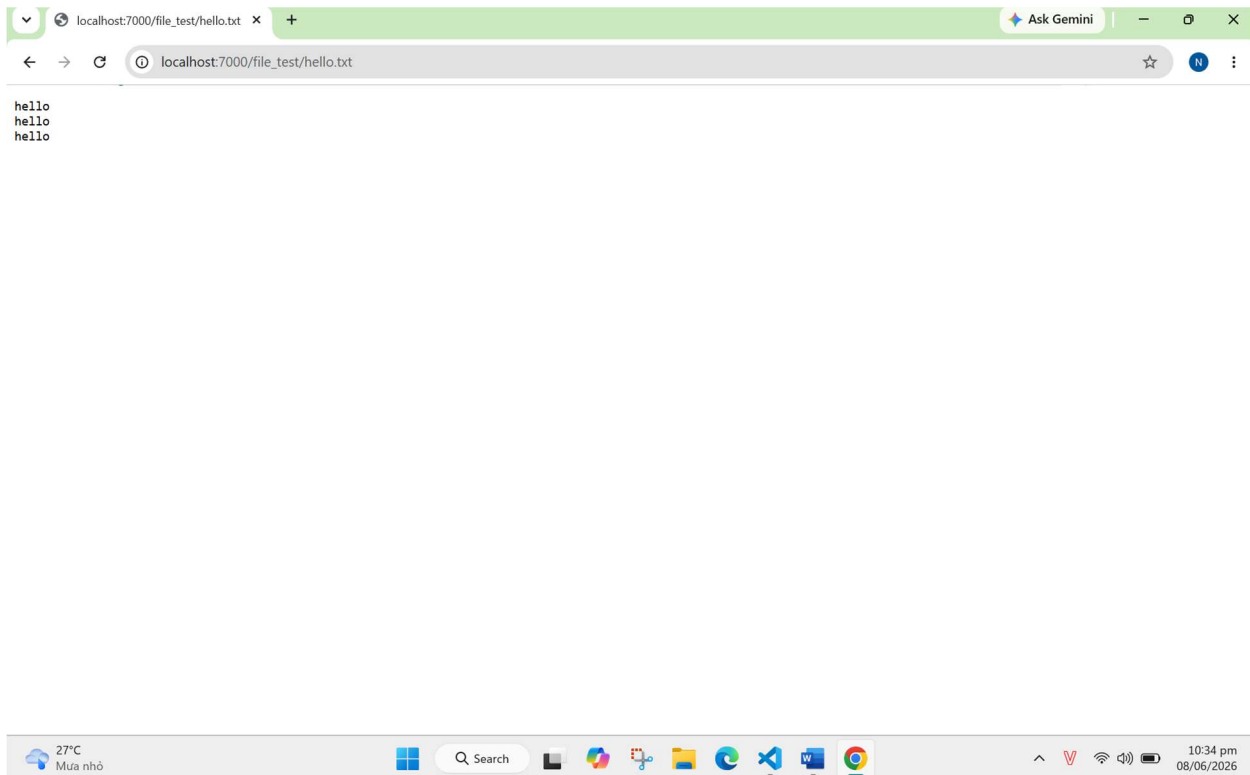
khởi chạy server lắng nghe kết nối ở cổng 7000



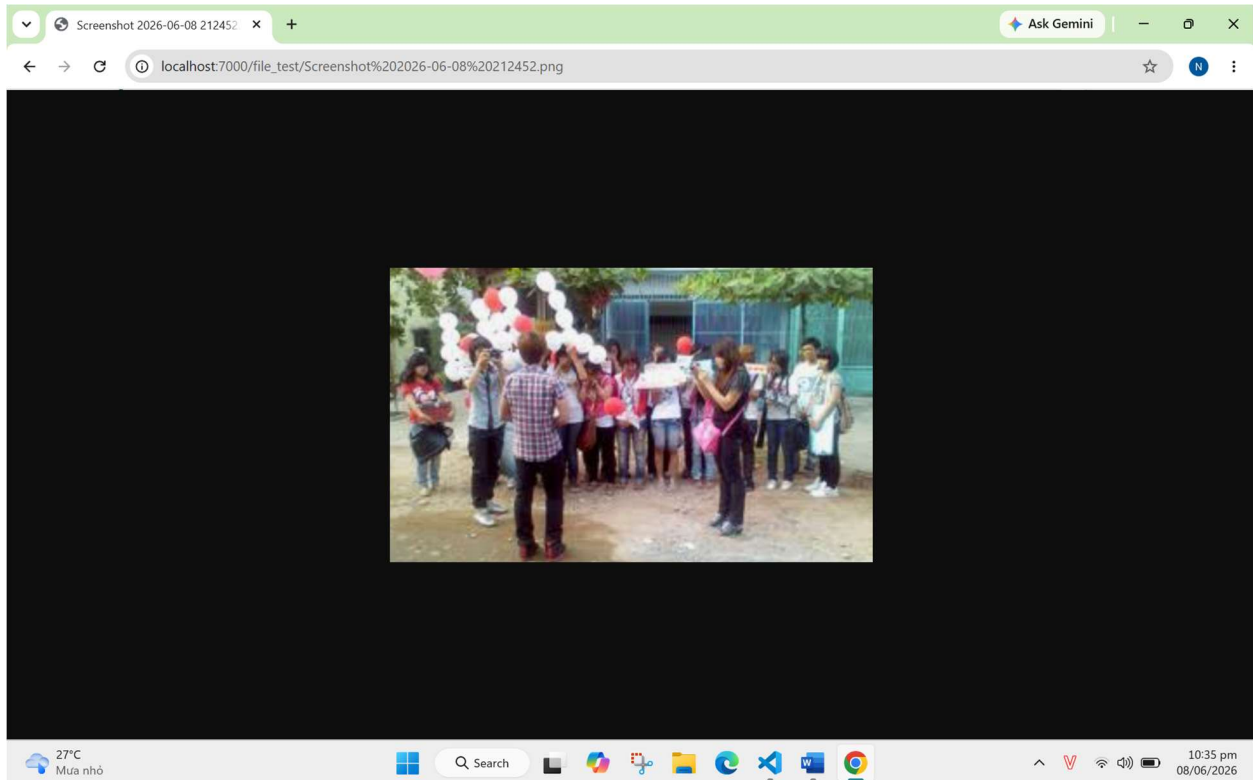
kết quả thư mục gốc trên trình duyệt được hiển thị



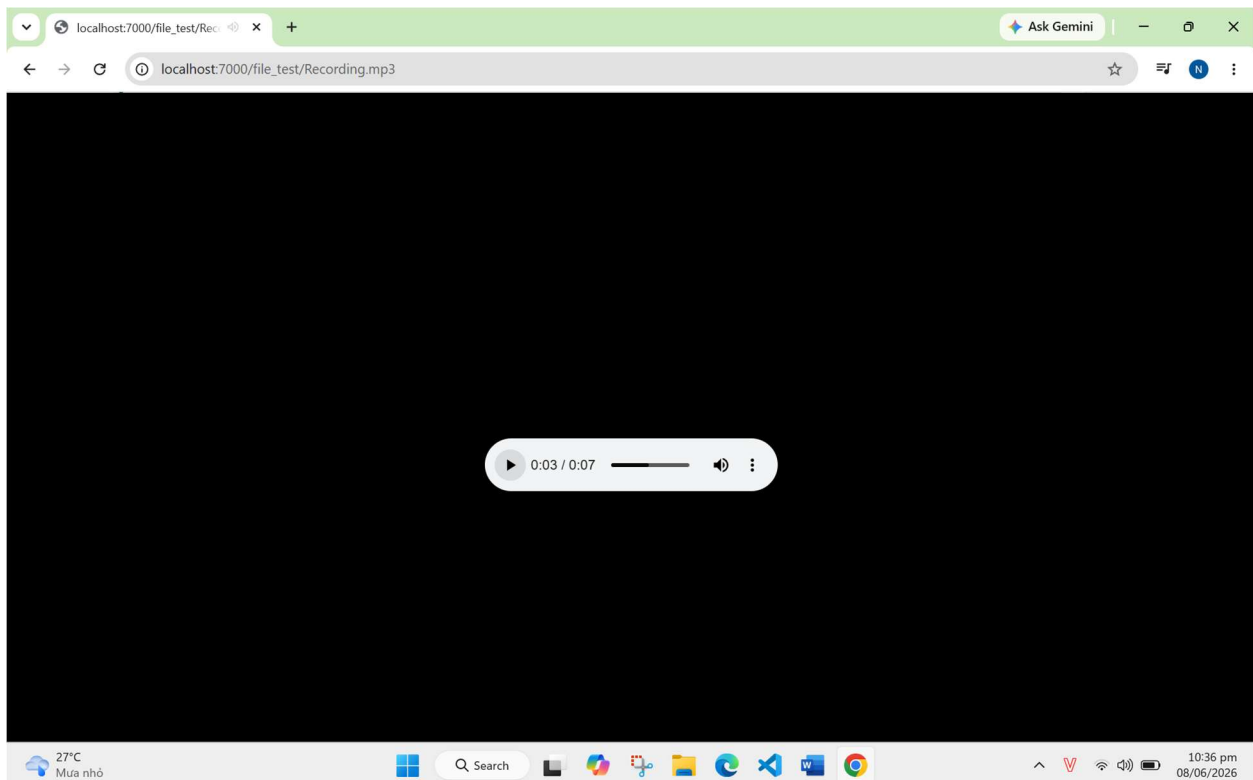
folder file_test



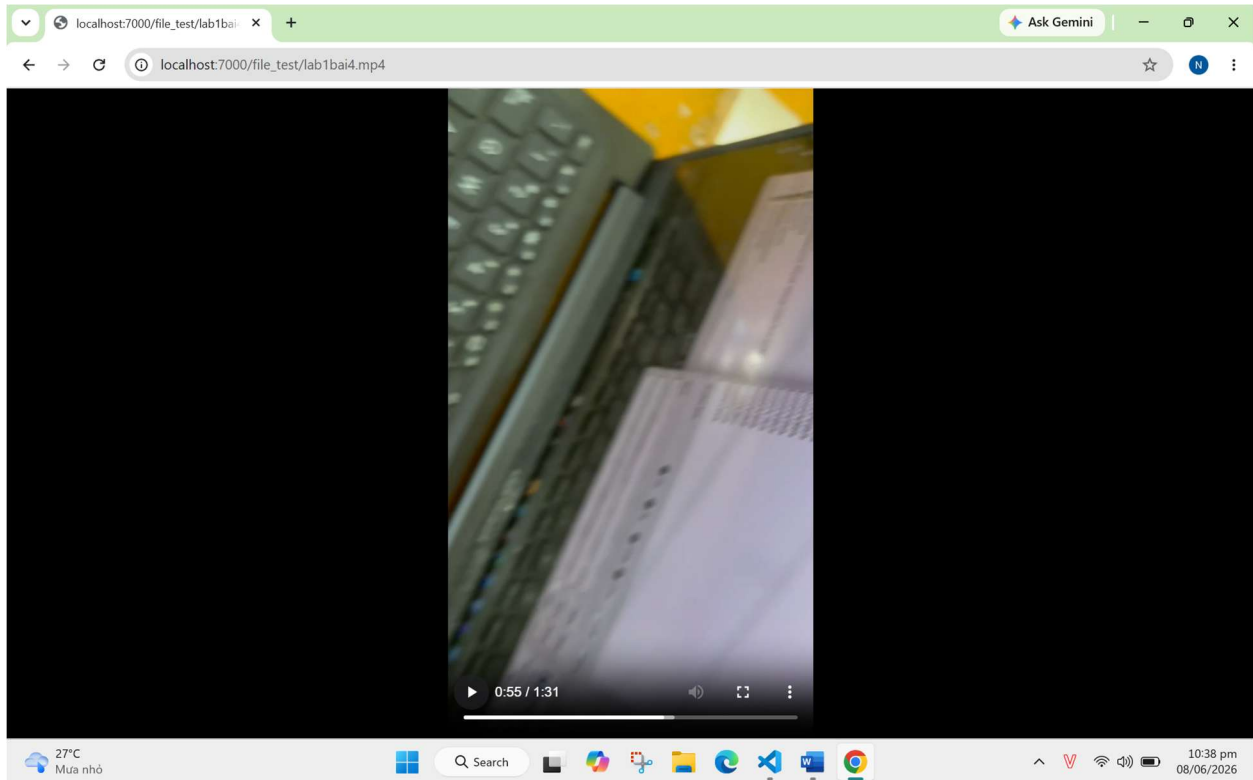
file hello.txt



file Screenshot 2026-06-08 212452.png



file Recording.mp3



file lab1bai4.mp4